

Agentic AI Engineer Pathway — Capstone Hackathon

Customer Service & Retention Copilot

Business Case BC-AAIE-HACK-17 · Domain: Telecom · Cross-cutting finale: Agentic Core + Context Engineering + MCP + Observability + Cost Governance + Security & Governance + Agent Evaluation

1. Project Identity

Business Case Title	Customer Service & Retention Copilot
Business Case ID	BC-AAIE-HACK-17
Domain	Telecom
Project Type	Agentic AI Engineer Pathway — Cross-Cutting Capstone Hackathon

2. Engagement Overview

Duration	20 hours
Format	Team of 2–4
Evaluation Mode	Automated review of the submitted Git repository against the Hackathon Rubric (7 categories / 100 marks), scored entirely from committed evidence in the repository. No live demo judging.
Submission	Push the final repository to your assigned Virtusa GitLab project by the cut-off.
Review Output	Per-team Excel report (Summary, Categories, Scorecard, Detailed, Improvement).
Grade Bands	Pass ≥ 60 · Not Yet Passed < 60

What is evaluated: a working LangGraph multi-agent system (foundation: context engineering, tiered memory, a custom MCP server, an agentic-RAG tool) that is then instrumented and governed — Arize Phoenix observability, cost & latency governance, guardrails & audit, governance/compliance documentation, and agent-level evaluation & testing. Every claim is scored from committed evidence.

What is not evaluated: the visual polish of any interface, generic unit-test volume, or which optional deployment path you use. Containerized/cloud deployment is out of scope for this cut.

3. Problem Statement & Expected Solution

3.1 Problem

A telecom care agent handling a customer contact must understand the intent (billing query, plan change, complaint, cancellation), pull the account and plan, check retention and offer policy, and decide the best resolution — including a compliant retention offer for at-risk customers. Done manually, resolutions are inconsistent and churn is missed. The operator wants an agentic copilot that resolves a contact, retrieves the offer policy, and drafts a compliant resolution with a human gate on discounts and credits.

3.2 Your Role

Agentic AI Engineer. You build a LangGraph multi-agent copilot, then instrument it with Arize Phoenix, govern its cost and latency, harden it with guardrails and an audit trail, document its risk and compliance posture, and prove its behaviour with agent-level evaluation and tests — delivering every artifact as committed evidence.

3.3 Expected Solution

A working, instrumented multi-agent application delivered as a Git repository that demonstrates:

- A LangGraph graph: typed state, a supervisor routing customer contacts to specialized worker agents (an intent-classification agent, a retention-offer agent, an offer-policy retrieval agent), conditional routing, checkpointing and structured output.
- A custom MCP server (≥ 2 tools + 1 resource) consumed via langchain-mcp-adapters, engineered context (write/select/compress/isolate + summarization + quarantine of untrusted customer-supplied text), tiered memory with verified cross-session persistence, and an agentic-RAG tool over a synthetic offer-and-retention policy corpus.
- Arize Phoenix instrumentation with a committed trace export, a machine-generated tool-invocation log, and an evidence-linked failure-mode analysis.
- A Phoenix-derived golden-signals report and a cost/latency dashboard; input/output guardrails, an audit trail and secrets hygiene; a governance pack (risk register, model card, compliance mapping, output-risk classification).
- Agent-level evaluation (LLM-as-judge + hallucination) and agent tests (routing-logic, loop/cascade guard, tool-contract), with a reproducible local-run runbook.

3.4 Applicable Rules

- **Evidence-in-Repo Rule.** Only committed artifacts are scored. A claim with no committed evidence scores zero; an evidence artifact with no producing code (a metric, trace or log a team could hand-write) is heavily discounted.
- **Citation-Resolves Rule.** Wherever a document cites evidence (a Phoenix run/span id, a log record, a control file), the citation must resolve to a committed artifact. Unresolvable citations are treated as missing.
- **Synthetic-Data Rule.** Use only synthetic accounts, plans and offer policies you generate. Customer identifiers and account/billing details must be synthetic and masked where shown; never write them to logs in plaintext.
- **Open-Source & Gemini-Only Rule.** Use the approved open-source stack with Google Gemini as the only model provider (not Claude). Build, run and evaluate with pip + Python — no Docker or external database service required for this cut.
- **Reproducibility Rule.** The system, its traces and its evaluation must be regenerable from a single documented command with committed sample inputs.

4. Technology & Framework Stack

Fixed open-source toolchain with Google Gemini as the only model provider. Everything installs with pip; no Docker or external DB service required.

Layer	Approved tool (open source unless noted)
Language / Agent Framework	Python 3.11+ · LangGraph (MIT)
LLM Provider	Google Gemini (API) — the only approved provider; not Claude
Interoperability	MCP Python SDK (stdio) + langchain-mcp-adapters
Memory	langgraph-checkpoint-sqlite (SQLite file) + LangMem
Retrieval	Chroma or FAISS + Sentence-Transformers (local)
Observability (mandated)	Arize Phoenix + OpenTelemetry / openinference (local, in-process)
Evaluation	DeepEval (LLM-as-judge = Gemini) · pytest for agent tests
Security	Guardrails-AI / LLM Guard · Presidio (PII) · python-dotenv
Interface	CLI (required) · FastAPI streaming (optional / bonus)

5. Acceptance Criteria & Non-Functional Requirements

5.1 Functional Acceptance Criteria

ID	Criterion
AC-01	Given a synthetic customer contact, the copilot classifies the intent, pulls the account/plan, and retrieves the applicable offer policy, citing it.
AC-02	For an at-risk customer it selects a compliant retention offer within policy limits and blocks any offer that would exceed a policy threshold.
AC-03	The copilot drafts the resolution (resolve / offer / escalate); a credit or discount above a threshold is routed for human approval, not auto-granted.
AC-04	The copilot identifies each contact's intent and handles it with the right capability; ambiguous or out-of-scope requests are clarified or escalated to a human, not mishandled.
AC-05	The copilot maintains context — it uses facts stated earlier in the interaction and recalls prior-session context on a return visit.
AC-06	The copilot handles untrusted customer-supplied input safely: attempts to inject instructions or access another customer's data are refused, and sensitive data (customer identifiers, account and billing details) is never exposed in answers or logs.
AC-07	A machine-generated tool-invocation log (logs/tool_calls.jsonl) written by committed logging middleware; tool names reconcile with the agent/MCP code.
AC-08	docs/failure-analysis.md documents ≥ 3 real failures from your own runs, each citing the Phoenix run_id + span_id (or a tool-log record) that shows it, plus root cause and fix.
AC-09	A Phoenix-derived golden-signals report (latency incl. thinking/acting/tool, tokens, cost estimate, plus accuracy + hallucination rate from the eval) and a cost/latency dashboard (Phoenix screenshot + underlying data file).
AC-10	Input/output guardrails wired into the agent's I/O path, and a machine-generated audit trail of agent actions (logs/agent_actions.jsonl).
AC-11	A governance pack: risk register, model/system card, compliance mapping (EU AI Act / NIST AI RMF / DPDP) and output-risk classification — each mitigation/claim citing a committed control.
AC-12	Agent evaluation: a DeepEval (or equivalent) report over a golden set (hallucination + faithfulness/relevance), and agent tests — routing-logic, loop/cascade guard, and tool-contract.

5.2 Non-Functional Requirements

ID	Requirement
NFR-01	No secrets/keys committed; env-var config with a committed .env.example and a .gitignore covering .env.
NFR-02	Single documented command runs the copilot and a second regenerates the Phoenix traces and the evaluation; committed sample inputs.
NFR-03	Untrusted free-text customer-supplied content is quarantined and never treated as instructions.
NFR-04	Agent pipeline uses async where it calls tools/models; graceful degradation on tool/model failure (timeouts, retries, exit conditions).
NFR-05	All data synthetic; customer identifiers, account and billing details masked and never logged in plaintext.
NFR-06	Evidence artifacts (traces, logs, reports) are machine-generated by committed code; the code that produced each is committed alongside it.

6. Functional Scope

6.1 In Scope

- The LangGraph multi-agent copilot (foundation) plus its full observability, cost-governance, security, governance and evaluation surface.
- Arize Phoenix tracing, golden-signals + cost/latency governance, guardrails + audit + secrets hygiene, governance/compliance docs, and agent-level evaluation & tests.
- A CLI to drive contacts through the graph and to regenerate traces and the evaluation.

6.2 Out of Scope (this cut)

- Containerized / cloud deployment (Docker, Rancher, k8s) — deferred; do not spend hackathon time on it.

- Real CRM / billing integrations — use synthetic account and policy data.
- Front-end visual polish; generic unit-test volume for its own sake.
- Advanced OAuth flows and live secrets-rotation infrastructure (document the approach instead).

7. Required Artifacts — What to Commit

This is the checklist your submission is scored against. Each artifact must be present at (or near) the path shown and contain what is listed. Remember the Evidence-in-Repo and Citation-Resolves rules: outputs must be produced by committed code, and every citation must resolve to a committed artifact.

7.1 Agentic System — Foundation

Area	Artifact (path)	Must contain
LangGraph graph	<code>src/graph.py</code>	Typed state; supervisor + ≥3 worker agents; conditional edges; checkpointer; structured output at node boundaries
MCP server	<code>mcp_server/</code> + <code>logs/mcp_transcript.jsonl</code>	≥2 tools + 1 resource; consumed via langchain-mcp-adapters; committed tool-call transcript
Context engineering	<code>src/context/</code>	write/select/compress/isolate; summarization middleware; quarantine of untrusted text
Tiered memory	<code>src/memory/</code> + <code>tests/test_memory_persistence.py</code> + <code>logs/memory_test.log</code>	short + long/semantic memory; cross-session recall test with committed output log
Agentic-RAG tool	<code>src/tools/rag_tool.py</code> + <code>data/policy_corpus/</code>	retrieval-in-the-loop over a synthetic offer-and-retention policy corpus

7.2 Observability & Tracing (Arize Phoenix — mandated)

Area	Artifact (path)	Must contain
Phoenix instrumentation	<code>src/observability/tracing.py</code>	Phoenix/openinference tracer wired into the run path (called, not just imported)
Trace export	<code>traces/</code> <code>phoenix_spans.parquet</code> (or <code>.jsonl</code>)	≥1 full run; spans across multiple agents + every tool call; latencies present
Tool-invocation log	<code>logs/tool_calls.jsonl</code>	machine-generated; per call: timestamp, agent/node, tool_name, args, result, latency_ms, status; names reconcile with code
Failure-mode analysis	<code>docs/failure-analysis.md</code>	≥3 real failures, EACH citing Phoenix run_id + span_id (or tool-log record) + root cause + fix

7.3 Performance & Cost Governance

Area	Artifact (path)	Must contain
Golden-signals report	<code>reports/golden_signals.json</code> + producing script	Phoenix-derived latency (thinking/acting/tool), tokens in/out, cost estimate; accuracy + hallucination rate from the eval
Cost/latency dashboard	<code>reports/dashboard.png</code> + <code>reports/dashboard_data.csv</code>	Phoenix latency/cost/token dashboard screenshot AND the underlying data file it was drawn from

7.4 Security & Guardrails

Area	Artifact (path)	Must contain
Guardrail code	<code>src/guardrails/</code>	input/output guardrails wired into the agent's I/O path (blocks/sanitizes)
Audit trail	<code>logs/agent_actions.jsonl</code> + audit middleware	machine-generated: actor, action, tool, decision, timestamp for consequential actions
Secrets hygiene	<code>.env.example</code> , <code>.gitignore</code>	env-var config; <code>.gitignore</code> covers <code>.env</code> ; no secrets committed anywhere

7.5 Governance & Compliance (citation-gated)

Area	Artifact (path)	Must contain
Risk register	<code>docs/risk-register.md</code>	risk, category (OWASP/NIST), likelihood, impact, mitigation (cite the committed control), residual risk, owner

Area	Artifact (path)	Must contain
Model / system card	<code>docs/model-card.md</code>	model (Gemini), data (synthetic), intended use, limitations, known failure modes (cite failure-analysis.md), out-of-scope
Compliance mapping	<code>docs/compliance.md</code>	applicable EU AI Act / NIST AI RMF / DPDP obligations → how addressed → evidence artifact
Output-risk classification	<code>docs/output-risk.md</code>	low/med/high output tiers + how high-risk is gated (human-in-loop / refusal) + a sample

7.6 Agent Evaluation & Testing (lean, agent-specific)

Area	Artifact (path)	Must contain
Evaluation report	<code>reports/eval_report.json</code> + harness	DeepEval (or equiv) over a golden set: hallucination + faithfulness/relevance; LLM-as-judge
Routing-logic test	<code>tests/test_routing.py</code>	asserts conditional edges route the right worker for given states
Loop/cascade guard	<code>tests/test_loops.py</code>	asserts a max-steps / recursion-limit stops runaway loops
Tool-contract test	<code>tests/test_tool_contracts.py</code>	asserts each tool's input/output schema + one error path

7.7 Engineering & Delivery

Area	Artifact (path)	Must contain
Local-run runbook	<code>README.md</code>	single-command run; how to regenerate traces (Phoenix) and the eval; committed sample inputs
Bonus	<code>src/api/</code> (FastAPI streaming)	OPTIONAL: async FastAPI streaming endpoint — extra credit, not required

8. Producing the Evidence — Format & Where to Obtain

Every evidence artifact must be produced by committed code and committed in the format shown below. This tells you the exact format and the tool / method to generate each one, so “observability” or “cost governance” becomes a concrete file you generate — not a slide. Enable Arize Phoenix locally (`pip install arize-phoenix openinference-instrumentation-langchain`; the UI runs at localhost:6006) — it is the single source your traces, latency, token and cost evidence come from.

Artifact	Format	How to produce it / where to get it
Phoenix trace export	Parquet or JSONL of OTEL spans	Turn on Phoenix tracing (<code>openinference-instrumentation-langchain</code>), run one full conversation, then export: <code>px.Client().get_spans_dataframe().to_parquet('traces/phoenix_spans.parquet')</code> .
Tool-invocation log	JSONL — one object per tool call	A logging wrapper/decorator on every tool that appends <code>{timestamp, agent, tool_name, args, result, latency_ms, status}</code> to <code>logs/tool_calls.jsonl</code> .
Failure-mode analysis	Markdown	Open your Phoenix traces, pick ≥ 3 real failures, write each up citing its <code>run_id</code> + <code>span_id</code> , with root cause and the fix.
Golden-signals report	JSON	A script reads the Phoenix spans (<code>get_spans_dataframe()</code>): p50/p95 latency by span type (thinking / acting / tool), token totals from LLM spans, cost = tokens × price; import accuracy + hallucination from the eval report; write <code>reports/golden_signals.json</code> .
Cost/latency dashboard	PNG + CSV	Phoenix UI (localhost:6006) shows latency / cost / token dashboards — screenshot one; export the data with <code>get_spans_dataframe().to_csv('reports/dashboard_data.csv')</code> .
Guardrail code	Python module	Guardrails-AI / LLM Guard validators (or policy functions) wrapped around the agent's input and output, wired into the graph's I/O nodes.
Audit trail	JSONL	Audit middleware that appends <code>{actor, action, tool, decision, timestamp}</code> for each consequential action to <code>logs/agent_actions.jsonl</code> .
Governance pack	Markdown	<code>risk-register.md</code> , <code>model-card.md</code> , <code>compliance.md</code> , <code>output-risk.md</code> — each entry cites the committed control/artifact it refers to.
Evaluation report	JSON	A DeepEval run over your golden set (hallucination, faithfulness, answer-relevance) that writes <code>reports/eval_report.json</code> , plus the

Artifact	Format	How to produce it / where to get it
		harness.
Agent tests	pytest files	<code>tests/test_routing.py</code> (routing), <code>tests/test_loops.py</code> (recursion/step limit), <code>tests/test_tool_contracts.py</code> (I/O schema + error path).

8.1 Good-to-Have

- FastAPI streaming endpoint; a demonstrated local run (screenshot/log).
- PII-redaction middleware (Presidio) with a before/after sample; a small red-team attack set + results.
- An optimization note showing a measured before/after latency or cost improvement (two Phoenix-derived reports).

On completion you will have demonstrated the skill the industry actually screens for: not just building a LangGraph agent, but making a service resolution observable, cost-governed, secure, compliant and continuously evaluated — the full production surface of an agentic system, proven with committed evidence rather than a demo that worked once.